CADETX VIRTUAL WORK EXPERIENCE

SPRINT 1 DATA PROFILING FINDINGS

Project: Heavy Supplier, Inventory & Warehouse Analytics
Prepared by: Keren Oyem
Role: Data Analyst
Task: S1-12

1. Purpose and Scope

The purpose of this Sprint 1 task was to profile the available warehouse, supplier, inventory, customer, purchasing, sales, invoice, payment, and stock movement datasets.
The profiling focused on:

- Understanding the structure and size of each table.
- Reviewing column names and data types.
- Identifying missing values.
- Identifying duplicate records.
- Highlighting potential data-quality issues.
- Identifying additional validation checks required before analysis and data modelling.

Data Profiling Review — All 12 Tables

1.1. Overall Profiling Summary

Table	Rows	Columns	Missing values	Duplicate row patterns
Branches	6	13	None	0
Products	30	23	None	0
inventory_master	180	8	None	0
Suppliers	8	12	None	0
customers	500	15	None	0
Invoices	18,033	10	None	0
Payments	19,257	5	None	0
purchase_orders_header	24,000	10	received_date: 2,370	0
purchase_orders_lines	155,495	9	None	0
sales_orders_header	20,000	11	None	0
sales_orders_lines	130,402	9	None	0
stock_ledger	237,230	9	None	0

2. Detailed Profiling

2.1. branches

Profile: 6 rows × 13 columns

Data type observations

- branch_id – VARCHAR: identifier stored as text.
- branch_name, city, state, region, warehouse_type – VARCHAR: categorical/text fields.
- warehouse_capacity – **VARCHAR**: capacity is stored as text even though it may be needed for numerical calculations.
- service_center_available – BOOLEAN: appropriate for a yes/no field.
- manager_id, total_employees, avg_monthly_revenue, monthly_operational_cost, market_demand_index – BIGINT.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- **warehouse_capacity should be investigated and standardised** if it contains numeric capacity values.
- Check for negative or unrealistic employee counts.
- Check revenue and operational-cost values for unrealistic values.
- Check that branch_id is unique.
- Validate manager_id against the relevant manager/employee information if available.

2.2. products

Profile: 30 rows × 23 columns

Data type observations

- product_id, product_name, category, machine_type, brand, model_compatibility, dimensions_cm, material_type, criticality_level, usage_frequency, uom – VARCHAR.
- unit_cost, unit_price, gst_rate, warranty_months, reorder_level, safety_stock, max_stock_level, lead_time_days, last_purchase_price – BIGINT.
- margin_percentage, weight_kg – DOUBLE.
- last_purchase_date – DATE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- **dimensions_cm is VARCHAR** and should be checked for consistent formatting.
- Financial fields use different numeric types.
- Check whether unit_price >= unit_cost.
- Check whether:

Safety Stock ≤ Reorder Level ≤ Maximum Stock Level

- Check margin_percentage and gst_rate for valid ranges.
- Check lead_time_days for negative/unrealistic values.

2.3. inventory_master

Profile: 180 rows × 8 columns

Data type observations

- product_id, branch_id, warehouse_bin – VARCHAR.
- opening_stock, reorder_level, safety_stock, max_stock, current_stock – BIGINT.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Check whether (product_id, branch_id) is unique.
- Check for negative stock values.
- Check:

Safety Stock ≤ Reorder Level ≤ Max Stock

- Check that product_id exists in products.
- Check that branch_id exists in branches.
- Check whether warehouse_bin follows a consistent format.

2.4. suppliers

Profile: 8 rows × 12 columns

Data type observations

- supplier_id, supplier_name, supplier_type, product_category, city, province, region, china_tax_id – VARCHAR.
- pincode, lead_time_days, reliability_score, import_duty_rate – BIGINT.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- **pincode is BIGINT.** This should be reviewed because postal codes can contain leading zeros.
- Check reliability_score range.
- Check import_duty_rate range.
- Check lead_time_days is non-negative.
- Check supplier_id uniqueness.
- Validate china_tax_id format.

2.5. customers

Profile: 500 rows × 15 columns

Data type observations

- customer_id, customer_type, industry_segment, city, state, region, branch_id, payment_terms – VARCHAR.
- pincode, credit_limit, current_balance, total_purchase_value, customer_rating – BIGINT.
- customer_since, last_purchase_date – DATE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Review pincode as BIGINT.
- Check credit_limit, current_balance, and total_purchase_value for unusual values.
- Check customer_rating range.
- Check:

customer_since ≤ last_purchase_date

- Check customer_id uniqueness.
- Validate branch_id against branches.

2.6. invoices

Profile: 18,033 rows × 10 columns

Data type observations

- invoice_id, so_id, customer_id, branch_id, payment_status – VARCHAR.
- invoice_date, due_date – DATE.
- total_order_value – BIGINT.
- total_gst_amount, grand_total – DOUBLE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Monetary fields use **mixed numeric types**.
- Consider standardising financial fields to a consistent decimal type during transformation.
- Check:

invoice_date ≤ due_date

- Check whether:

grand_total = total_order_value + total_gst_amount

subject to the project's exact financial rules.

- Check valid payment_status values.
- Validate so_id, customer_id, and branch_id.

2.7. payments

Profile: 19,257 rows × 5 columns

Data type observations

- payment_id, invoice_id, payment_method – VARCHAR.
- payment_date – DATE.
- payment_amount – DOUBLE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- payment_amount is DOUBLE; consider DECIMAL for financial reporting.
- Check for negative payment amounts.
- Check payment dates against invoice dates.
- Check invoice_id against invoices.
- Investigate invoices with multiple payments where relevant.

2.8. purchase_orders_header

Profile: 24,000 rows × 10 columns

Data type observations

- po_id, supplier_id, branch_id, po_status – VARCHAR.
- order_date, expected_delivery_date, received_date – DATE.
- total_cost, total_gst_amount, grand_total – DOUBLE.

Data quality issues / checks

- **received_date has 2,370 missing values.**
- These missing values may be legitimate if the PO has not yet been received, so this needs business-rule validation rather than automatic filling.

- No duplicate row patterns.
- Monetary fields use DOUBLE.
- Check:

order_date ≤ expected_delivery_date

- Check received_date against order_date and expected_delivery_date.
- Check whether po_status agrees with received_date.
- Check:

grand_total = total_cost + total_gst_amount

subject to the project's calculation rules.

- Validate supplier_id and branch_id.

2.9. purchase_orders_lines

Profile: 155,495 rows × 9 columns

Data type observations

- po_id, product_id – VARCHAR.
- line_number, quantity, gst_rate – BIGINT.
- unit_cost, line_total, gst_amount, line_grand_total – DOUBLE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Check uniqueness of:

(po_id, line_number)

- Check that quantity is positive.
- Check:

line_total ≈ quantity × unit_cost

- Validate GST calculations.
- Validate line_grand_total.
- Check po_id against purchase_orders_header.
- Check product_id against products.

2.10. sales_orders_header

Profile: 20,000 rows × 11 columns

Data type observations

- so_id, customer_id, branch_id, order_status, payment_terms, sales_channel – VARCHAR.
- order_date, delivery_date – DATE.
- total_order_value – BIGINT.
- total_gst_amount, grand_total – DOUBLE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Mixed monetary data types.
- Check:

order_date ≤ delivery_date

- Check:

grand_total = total_order_value + total_gst_amount

subject to business rules.

- Validate order_status.

- Validate payment_terms.
- Validate sales_channel.
- Check customer_id against customers.
- Check branch_id against branches.

2.11. sales_orders_lines

Profile: 130,402 rows × 9 columns

Data type observations

- so_id, product_id – VARCHAR.
- line_number, quantity, unit_price, gst_rate, line_total – BIGINT.
- gst_amount, line_grand_total – DOUBLE.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Check uniqueness of:

(so_id, line_number)

- Check for zero or negative quantities.
- Check:

line_total ≈ quantity × unit_price

- Validate GST calculation.
- Validate line_grand_total.
- Check so_id against sales_orders_header.
- Check product_id against products.

2.12. stock_ledger

Profile: 237,230 rows × 9 columns

Data type observations

- movement_id, product_id, branch_id, movement_type, reference_type, reference_id – VARCHAR.
- movement_date – DATE.
- quantity, running_balance – BIGINT.

Data quality issues / checks

- No missing values.
- No duplicate row patterns.
- Validate movement_type.
- Validate reference_type.
- Check movement_id uniqueness.
- Check whether the sign of quantity correctly represents inbound/outbound movement.
- Check **running-balance continuity** by product and branch.
- Validate product_id against products.
- Validate branch_id against branches.
- Validate reference_id against the relevant transaction table where applicable.

3. Cross-Table Issues

1. Data-type standardisation

The most obvious fields to investigate are:

Field	Table	Observation
warehouse_capacity	branches	VARCHAR but potentially numeric
dimensions_cm	products	VARCHAR; format needs standardisation
Pincode	suppliers/customers	BIGINT; may be better as VARCHAR
Monetary fields	several transaction tables	BIGINT and DOUBLE are mixed

2. Referential integrity

You should check relationships such as:

inventory_master.product_id → products.product_id
inventory_master.branch_id → branches.branch_id
customers.branch_id → branches.branch_id
invoices.customer_id → customers.customer_id
invoices.so_id → sales_orders_header.so_id
payments.invoice_id → invoices.invoice_id
purchase_orders_header.supplier_id → suppliers.supplier_id
purchase_orders_lines.po_id → purchase_orders_header.po_id
purchase_orders_lines.product_id → products.product_id
sales_orders_lines.so_id → sales_orders_header.so_id
sales_orders_lines.product_id → products.product_id
stock_ledger.product_id → products.product_id
stock_ledger.branch_id → branches.branch_id

3. Business-rule validation

You should check:

- Stock-level relationships
- Negative quantities
- Order and delivery dates
- PO and received dates
- Invoice and due dates
- Payment dates
- GST calculations
- Grand-total calculations
- Line-total calculations
- Running stock balances

4. Missing data

The main missing-data finding from the profiling is:

purchase_orders_header.received_date — 2,370 missing values.

4. Key Findings

1. **All 12 tables were successfully profiled.**
2. **No duplicate row patterns were detected across the 12 tables.**
3. Most tables have **zero missing values**.
4. purchase_orders_header.received_date has **2,370 missing values** and requires investigation.
5. warehouse_capacity and dimensions_cm require data-type/format review.
6. Postal codes in supplier and customer tables are stored as BIGINT and should be reviewed.
7. Financial fields use inconsistent numeric types across tables.
8. Several relationships require **foreign-key validation** before building the analytical model.
9. Transaction-level calculated fields should undergo **financial reconciliation**.
10. Stock movements should be checked against running balances and inventory records.

5. Recommended Actions

Priority	Action
High	Investigate the 2,370 missing received_date values
High	Perform foreign-key/referential-integrity checks
High	Reconcile sales, purchase, invoice and payment totals
Medium	Standardise monetary data types
Medium	Standardise warehouse_capacity
Medium	Parse/standardise dimensions_cm
Medium	Validate stock-level business rules
Low	Review postal-code data types
Low	Validate categorical fields against allowed values